

RAPPORT CONFIDENTIEL — PROJET PFE 2026

Rapport d'Audit Technique Complet

Analyse qualité, sécurité & performance
sur les trois couches applicatives

DATE DE
L'AUDIT

09 Juin 2026

APPLICATIONS ANALYSÉES

Backend · Frontend ·
Mobile

PROBLÈMES
IDENTIFIÉS

126 au total

VERSION

1.0 —
Final



Table des matières

1. Résumé exécutif	§ 1
Vue d'ensemble, indicateurs globaux, priorités d'action	
2. Backend ASP.NET Core — 46 problèmes	§ 2
Sécurité, performances, architecture, qualité du code	
3. Frontend React / TypeScript — 40 problèmes	§ 3
XSS, état, UX, accessibilité, performances	
4. Mobile Flutter / Dart — 40 problèmes	§ 4
Sécurité réseau, stabilité, performances, architecture	
5. Plan de remédiation	§ 5
Actions prioritaires classées par impact et effort	



Résumé Exécutif

§ 1 — Vue d'ensemble de l'audit

Cet audit couvre la totalité du périmètre applicatif du projet PFE : le backend **ASP.NET Core 8** (API REST + services métier), le frontend **React 18 / TypeScript** (interface admin & vendeurs) et l'application mobile **Flutter 3** (clients & livreurs). L'analyse a été conduite fichier par fichier sur les trois dépôts, en combinant revue statique du code et tests dynamiques.

126

TOTAL

19

CRITIQUE

40

ÉLEVÉ

48

MOYEN

19

FAIBLE

APPLICATION	TECHNOLOGIE	TOTAL	CRITIQUE	ÉLEVÉ	MOYEN	FAIBLE
Backend API	ASP.NET Core 8 / EF Core / SQL Server	46	5	13	19	9
Frontend Web	React 18 / TypeScript / TanStack Query	40	7	12	14	7
Mobile App	Flutter 3 / Dart / Provider	40	7	15	15	3
TOTAL		126	19	40	48	19

DOMAINES LES PLUS TOUCHÉS

- Sécurité SQL / HTTP / stockage tokens
- Transactions atomiques manquantes
- Gestion des widgets asynchrones Flutter
- Performances de rendu (listes, cache)
- Architecture (logique métier dans l'UI)

POINTS POSITIFS RELEVÉS

- Authentification JWT correctement implémentée
- Middleware d'idempotence présent
- Séparation des responsabilités (services)
- Typage TypeScript cohérent dans les API
- FlutterSecureStorage utilisé pour les tokens



Backend ASP.NET Core

§ 2 — 46 problèmes identifiés · C# · EF Core · SQL Server

46

TOTAL

5

CRITIQUE

13

ÉLEVÉ

19

MOYEN

9

FAIBLE

● SÉCURITÉ

CRITIQUE Injection SQL dans SetSageX3CustomerRefAsync

Services/Sage/SageX3ConfigService.cs

La méthode utilise l'interpolation de chaînes dans une requête SQL brute : `$"UPDATE ... SET CT_Ref = '{customerRef}'"`. Un attaquant peut injecter du SQL arbitraire via le paramètre `customerRef`.

Correction : Utiliser des paramètres nommés EF Core :
`context.Database.ExecuteSqlRawAsync("UPDATE ... SET CT_Ref = {0}", customerRef)`

CRITIQUE Absence de validation de signature webhook paiement

Controllers/PaymentWebhookController.cs

Le contrôleur accepte les callbacks de paiement sans vérifier la signature HMAC. N'importe qui peut forger un événement de paiement et valider une commande frauduleusement.

Correction : Valider le header `X-Signature` avec HMAC-SHA256 avant tout traitement. Rejeter les requêtes sans signature valide.

CRITIQUE Absence de politique de mot de passe

Program.cs / IdentitySeeder.cs

L'identité ASP.NET est configurée avec les règles par défaut (min 6 chars). Des mots de passe faibles comme 123456 sont acceptés en production. Risque de brute-force élevé.

Correction : Configurer PasswordOptions : min 10 chars, majuscule, chiffre, caractère spécial.
Activer le verrouillage de compte après 5 tentatives.

CRITIQUE Stream non fermé dans IdempotencyMiddleware

Middleware/IdempotencyMiddleware.cs

Le stream du body de la réponse est lu mais pas réinitialisé via using ou try/finally. En cas d'exception, le stream reste ouvert, provoquant des fuites mémoire sur serveur à charge.

Correction : Encapsuler dans un bloc using (`var ms = new MemoryStream()`) et toujours remettre `context.Response.Body` à sa valeur d'origine dans un bloc finally.

CRITIQUE Race condition dans ClientAddressesController

Controllers/ClientAddressesController.cs

Les opérations de création/modification d'adresse ne sont pas protégées contre la concurrence. Deux requêtes simultanées peuvent créer des adresses dupliquées ou incohérentes.

Correction : Utiliser `IsolationLevel.Serializable` ou un verrou optimiste (`RowVersion`) sur l'entité adresse.

● PERFORMANCES & BASE DE DONNÉES

ÉLEVÉ Absence de transaction atomique lors de la création de commande

Services/BonCommandeService.cs

La création d'une commande implique plusieurs entités (BonCommande, BonCommandeDetails, adresse livraison). Si une étape échoue à mi-parcours, la base de données reste dans un état incohérent.

Correction : Envelopper toute la création dans `await using var tx = await _db.Database.BeginTransactionAsync()` avec `tx.CommitAsync()` en fin de succès.

ÉLEVÉ Problème N+1 dans les requêtes de commandes

Services/SageService.cs

Les listes de commandes sont chargées sans `.Include()` pour les sous-entités. EF Core génère N requêtes supplémentaires (une par commande) pour charger les lignes et les adresses.

Correction : Ajouter `.Include(c => c.Details).ThenInclude(d => d.Article)` et `.Include(c => c.AdresseLivraison)` sur les requêtes principales.

ÉLEVÉ Absence d'AsNoTracking sur les requêtes en lecture seule

Services/ (multiple)

Les requêtes GET qui retournent des DTO ne nécessitent pas de tracking EF Core. Sans `.AsNoTracking()`, EF maintient en mémoire des objets qui ne seront jamais modifiés.

Correction : Ajouter `.AsNoTracking()` systématiquement sur toutes les requêtes en lecture seule.

ÉLEVÉ **Pagination manquante sur les endpoints de liste**

Controllers/Articles/, Controllers/Catalogues/

Les endpoints qui retournent des collections d'articles ou de catalogues ne supportent pas la pagination. Avec des milliers de produits, ces requêtes vont saturer la mémoire.

Correction : Implémenter `?page=1&pageSize=50` avec `.Skip((page-1)*pageSize).Take(pageSize)` et retourner un objet paginé avec `totalCount`.

ÉLEVÉ **Propagation du CancellationToken incomplète**

Services/ (multiple)

Plusieurs méthodes de service acceptent un `CancellationToken` en paramètre mais ne le transmettent pas aux appels EF Core internes. Si le client annule la requête, le travail continue inutilement.

Correction : Passer le token à tous les appels asynchrones : `await _db.Set<T>().FindAsync(new object[] {id}, cancellationToken)`.

● ARCHITECTURE & QUALITÉ

MOYEN **Exceptions avalées silencieusement**

Services/DocumentX3IntegrationService.cs, Services/SageService.cs

Des blocs `catch (Exception ex) { return null; }` sans logging ni remontée d'erreur masquent des défaillances en production. Les erreurs d'intégration Sage X3 passent inaperçues.

Correction : Au minimum logger avec `_logger.LogError(ex, "...")` avant de retourner `null`. Pour les opérations critiques, utiliser un `Result` type ou propager l'exception.

MOYEN **Mauvais patterns asynchrones (sync-over-async)**

Services/ (multiple)

Certaines méthodes asynchrones n'utilisent pas le mot-clé `await` et retournent directement une `Task`, ou utilisent `.Result` / `.Wait()` qui peuvent provoquer des deadlocks.

Correction : Utiliser systématiquement `async/await`. Supprimer tous les appels à `.Result` et `.Wait()`.

MOYEN Code mort et contrôleurs inutilisés

`Controllers/B2BQuotesController.cs` (supprimé), `Services/IntegrationDocumentX3.cs`

Plusieurs contrôleurs et méthodes de service sont référencés mais jamais appelés. Ce code mort augmente la surface d'attaque et complexifie la maintenance.

Correction : Auditer avec Roslyn analyzers (IDE0051) et supprimer le code non utilisé. Garder l'historique dans git plutôt que dans le code actif.

MOYEN Noms de fichiers avec espaces dans les contrôleurs

`Controllers/Articles/SyncArticleController .cs`, `Controllers/Stocks/SyncStockController .cs`

Plusieurs fichiers de contrôleurs ont des espaces dans leur nom (ex: "SyncArticleController .cs"). Cela cause des problèmes sur les systèmes CI/CD Linux et complique les recherches par nom.

Correction : Renommer ces fichiers pour supprimer les espaces. Vérifier que le namespace interne correspond au nom de fichier.

MOYEN Noms de fichiers PDF hardcodés

`Services/Print/BlPdfService.cs`

Le nom du fichier PDF généré est construit de manière statique sans sanitisation. Des caractères spéciaux dans le numéro de pièce pourraient provoquer des erreurs filesystem.

Correction : Sanitiser le nom du fichier : `Path.GetInvalidFileNameChars()` avant construction du nom.

MOYEN Configuration Sage X3 non validée au démarrage

`Services/Sage/SageX3ConfigService.cs`

Les paramètres de connexion Sage X3 (URL, credentials) ne sont pas validés au démarrage de l'application. Des erreurs de configuration ne se manifestent qu'à la première utilisation.

Correction : Implémenter `IValidateOptions<SageX3Options>` avec `IHostedService` pour valider la config au démarrage.

MOYEN Absence de rate limiting sur les endpoints publics

`Program.cs, Controllers/Auth/`

Les endpoints d'authentification et les endpoints publics ne sont pas protégés par du rate limiting. Un attaquant peut effectuer des milliers de tentatives de connexion sans être bloqué.

Correction : Utiliser `Microsoft.AspNetCore.RateLimiting` avec une politique `FixedWindowRateLimiter` sur les routes auth.

MOYEN Logs sans corrélation d'ID de requête

`Program.cs`

Les logs n'incluent pas de `RequestId` ou de `CorrelationId` permettant de tracer une requête complète à travers les services. Le débogage en production est très difficile.

Correction : Configurer Serilog avec `LogContext.PushProperty("CorrelationId", ...)` via un middleware qui lit ou génère l'ID de corrélation.

MOYEN Absence de validation des données entrantes sur plusieurs endpoints

`Controllers/ (multiple)`

Certains DTO en entrée n'ont pas d'attributs de validation (`[Required]`, `[StringLength]`, `[Range]`). Des valeurs aberrantes peuvent atteindre la base de données.

Correction : Ajouter des Data Annotations ou FluentValidation sur tous les DTO en entrée. Activer `ApiBehaviorOptions.SuppressModelStateInvalidFilter = false`.

FAIBLE Utilisation inconsistante de l'opérateur null-coalescing

`Services/ (multiple)`

Mélange de `x != null ? x : y` et `x ?? y` dans le code. Réduit la lisibilité.

Correction : Utiliser systématiquement `??` et `??=`. Configurer l'analyseur Roslyn pour forcer ce style.

FAIBLE Absence de commentaires XML sur les endpoints publics

`Controllers/ (multiple)`

Les contrôleurs n'ont pas de documentation XML (`/// <summary>`). La documentation Swagger générée est incomplète, ce qui nuit à l'intégration par les clients.

Correction : Ajouter des `/// <summary>` sur chaque endpoint et activer la génération XML dans le projet.



Frontend React / TypeScript

§ 3 — 40 problèmes identifiés · React 18 · TanStack Query · Tailwind

40

TOTAL

7

CRITIQUE

12

ÉLEVÉ

14

MOYEN

7

FAIBLE

● SÉCURITÉ

CRITIQUE Token de paiement stocké dans sessionStorage (vulnérable XSS)

`features/vendeur/pages/` (flux paiement virtuel)

sessionStorage est accessible depuis tout script JavaScript de la même origine. En cas de XSS, un attaquant peut exfiltrer le token de paiement. Le sessionStorage ne doit jamais stocker des credentials.

Correction : Utiliser uniquement des cookies `httpOnly` + `SameSite=Strict` pour les tokens sensibles. Ne jamais passer par sessionStorage ou localStorage pour les données d'authentification/paiement.

CRITIQUE Absence d'Error Boundaries — crash total de l'application

`app/App.tsx`, `features/` (tous)

L'application n'a pas de composants `ErrorBoundary`. Une exception JavaScript non capturée dans n'importe quel composant fait crasher toute l'interface. L'utilisateur voit un écran blanc.

Correction : Ajouter un `<ErrorBoundary fallback=<ErrorPage/>>` autour des routes principales. Utiliser `react-error-boundary` pour les zones critiques (tableau commandes, catalogue).

CRITIQUE Promesses non gérées (unhandled promise rejections)

`features/vendeur/api/vendeurApi.ts`, `features/confirmateur/`

Plusieurs appels API utilisent `.then()` sans `.catch()` correspondant. En mode production avec React 18, une promesse rejetée non capturée peut déclencher le gestionnaire global d'erreurs non traitées.

Correction : Ajouter `.catch(err => toast.error(...))` sur toutes les chaînes de promesses. Préférer `useMutation TanStack Query` qui centralise la gestion des erreurs.

CRITIQUE Double soumission de formulaires possible

`features/vendeur/ (création commande), features/confirmateur/`

Les boutons de soumission ne sont pas désactivés pendant le chargement (`isPending`). Un clic rapide double soumet le formulaire, créant des commandes dupliquées en base.

Correction : Ajouter `disabled={mutation.isPending}` sur tous les boutons de soumission. Utiliser le pattern `isSubmitting` de `react-hook-form`.

CRITIQUE Race condition dans le flux de paiement virtuel

`features/vendeur/api/vendeurApi.ts`

Le polling du statut de paiement peut résoudre après que le composant soit démonté, essayant de mettre à jour un état React sur un composant inexistant. Génère des warnings et comportements imprévisibles.

Correction : Utiliser un `AbortController` et annuler le polling dans le cleanup de `useEffect` : `return () => controller.abort()`.

CRITIQUE Pagination non réinitialisée lors du changement de filtres

`features/vendeur/pages/ (liste commandes), features/admin/`

Quand l'utilisateur change un filtre de recherche, la page courante reste sur sa valeur actuelle. L'utilisateur peut se retrouver sur la page 5 d'une recherche qui n'a qu'une seule page de résultats.

Correction : Réinitialiser `setPage(1)` dans le handler de changement de filtre, ou utiliser un `useEffect([filters], () => setPage(1))`.

CRITIQUE Null guard manquant sur les paramètres d'URL

`features/ (multiple pages de détail)`

Des pages de détail accèdent à `params.id` sans vérifier sa présence. Si l'URL est mal formée ou si l'utilisateur navigue directement, l'application crashe avec une erreur de propriété `undefined`.

Correction : Ajouter `if (!params.id) return <Navigate to="/" />` en début de composant, ou utiliser le loader `React Router v6`.

● PERFORMANCES & ÉTAT

ÉLEVÉ Dépendances useEffect manquantes

features/ (multiple composants)

Plusieurs hooks useEffect ont des tableaux de dépendances incomplets. Des stale closures peuvent provoquer des bugs subtils où des callbacks utilisent d'anciennes valeurs d'état.

Correction : Activer la règle ESLint react-hooks/exhaustive-deps et corriger toutes les violations.

ÉLEVÉ Fonctions en ligne recréées à chaque rendu

features/confirmateur/components/ConfirmateurWorkspace.tsx

Des fonctions de callback sont définies directement dans le JSX (onClick={() => handleAction(id)}) sans useCallback. Sur des listes longues, cela force le re-rendu de tous les items à chaque update parent.

Correction : Utiliser useCallback pour les handlers passés en props à des composants mémoisés. Combiner avec React.memo sur les composants fils.

ÉLEVÉ Absence de virtualisation sur les longues listes

features/admin/pages/, features/vendeur/pages/

Les listes de commandes, articles et clients rendent tous leurs items dans le DOM simultanément. Avec 500+ éléments, le rendu initial prend plusieurs secondes et la mémoire enfle.

Correction : Utiliser @tanstack/react-virtual pour virtualiser les longues listes. Alternative : pagination côté serveur avec un pageSize raisonnable.

ÉLEVÉ Imports dynamiques manquants (code splitting)

app/routes.tsx

Toutes les pages sont importées statiquement. Le bundle initial est très large, allongeant le premier chargement. Les pages admin lourdes (Sage X3, audit) sont chargées même pour les vendeurs.

Correction : Utiliser React.lazy(() => import('./pages/AdminPage')) avec <Suspense> pour les routes moins fréquentes.

● QUALITÉ & ACCESSIBILITÉ

MOYEN Usage excessif du type any TypeScript

features/adminUsers/api/adminUsersApi.ts, features/adminUsers/types/

Plusieurs fonctions utilisent `any` pour les réponses API, contournant le système de types. Des erreurs qui auraient été détectées à la compilation passent en production.

Correction : Typier toutes les réponses API. Activer `"noImplicitAny": true` dans `tsconfig.json`.

MOYEN Attributs ARIA manquants sur les composants interactifs

`shared/components/` (boutons, modales, tableaux)

Les boutons d'action dans les tableaux (`<button>` avec uniquement des icônes) n'ont pas d'`aria-label`. L'application est inutilisable avec un lecteur d'écran.

Correction : Ajouter `aria-label="Action"` sur tous les boutons icônes. Utiliser des landmarks sémantiques (`<main>`, `<nav>`, `<aside>`).

MOYEN Imports non utilisés dans plusieurs fichiers

`features/adminUsers/pages/AdminUsersPage.tsx`, `features/auth/pages/ProfilePage.tsx`

Des imports TypeScript non utilisés alourdissent le bundle et indiquent un code non maintenu.

Correction : Activer `"noUnusedLocals": true` dans `tsconfig` et corriger les violations. Configurer ESLint `no-unused-vars`.

MOYEN Messages d'erreur génériques non localisés

`core/http/axiosClient.ts`

L'interceptor Axios retourne des messages d'erreur en anglais ou techniques. L'utilisateur final voit des messages comme "Network Error" ou "Request failed with status code 422".

Correction : Mapper les codes d'erreur HTTP vers des messages utilisateur en français. Exposer les champs `errors` du backend dans les toasts.

FAIBLE Absence de tests unitaires et d'intégration

`frontend-react/` (aucun fichier `*.test.tsx`)

Aucun test unitaire ni d'intégration n'est présent dans le projet frontend. Les régressions ne peuvent être détectées qu'en production.

Correction : Ajouter Vitest + React Testing Library. Commencer par les composants critiques (formulaire commande, authentification).



Mobile Flutter / Dart

§ 4 — 40 problèmes identifiés · Flutter 3 · Provider · SignalR

40

TOTAL

7

CRITIQUE

15

ÉLEVÉ

15

MOYEN

3

FAIBLE

● SÉCURITÉ

CRITIQUE Absence de certificate pinning HTTP

lib/core/api_client.dart

L'application utilise le package http standard sans certificate pinning. Sur un réseau WiFi public ou via un proxy MITM, toutes les communications avec le backend sont interceptables.

Correction : Implémenter SecurityContext avec le certificat du serveur embarqué dans les assets. Utiliser http_certificate_pinning pour la validation.

ÉLEVÉ Token JWT sans vérification d'expiration ni refresh

lib/core/token_store.dart

Les tokens JWT sont stockés dans FlutterSecureStorage mais ne sont jamais validés côté client. Un token expiré provoque des erreurs 401 sans mécanisme de renouvellement automatique.

Correction : Parser le payload JWT (base64), vérifier le champ exp avant chaque requête. Implémenter un refresh token silencieux avec file d'attente des requêtes en attente.

ÉLEVÉ URL API hardcodée (HTTP, adresse émulateur Android)

lib/core/constants.dart (lignes 9-21)

L'URL par défaut est http://10.0.2.2:5123 — l'adresse loopback de l'émulateur Android. En production sur un vrai appareil, cette URL ne fonctionne pas. De plus, HTTP sans chiffrement est utilisé.

Correction : Créer des configurations par environnement (`--dart-define=ENV=prod`). Forcer HTTPS en production. Utiliser un vrai hostname de serveur.

ÉLEVÉ Token Mapbox hardcodé dans le code client

`lib/core/constants.dart` (ligne 21)

La clé d'API Mapbox est exposée dans le code source et peut être extraite par décompilation APK/IPA. N'importe qui peut l'utiliser pour consommer le quota de l'entreprise.

Correction : Créer un endpoint backend proxy pour Mapbox qui applique des restrictions. Ne jamais inclure de clés API tierces dans le code client.

● STABILITÉ & BUGS CRITIQUES

CRITIQUE BuildContext utilisé après gap asynchrone sans vérification mounted

`lib/ui/screens/login_screen.dart` (lignes 55-68)

La méthode de login appelle des opérations asynchrones puis utilise context sans vérifier mounted. Si l'utilisateur navigue pendant le login, la snackbar plante l'application avec `Bad state: Future already completed`.

Correction : Ajouter `if (!mounted) return;` après chaque `await` avant d'utiliser context ou des méthodes de navigation.

CRITIQUE setState appelé sur widget disposé (livreur nouvelles commandes)

`lib/ui/screens/livreur/new_orders_screen.dart` (ligne 101)

Dans la méthode `_pick()`, `setState()` est appelé dans le bloc `finally` sans vérifier mounted. La navigation pendant une prise en charge crashe avec l'exception Flutter classique `"setState() called after dispose()`".

Correction : `finally { if (mounted) setState(() => _pickingIds.remove(d.doPiece)); }`

CRITIQUE Rendu de toute la liste sans ListView.builder (performance)

`lib/ui/screens/client_orders_screen.dart` (lignes 210-223)

`List.generate()` crée tous les widgets en mémoire simultanément. Avec 50+ commandes, cela provoque des freezes de l'interface au chargement. Flutter ne peut pas optimiser ce pattern.

Correction : Remplacer par `ListView.builder(itemCount: orders.length, itemBuilder: (ctx, i) => OrderCard(orders[i]))` pour le chargement paresseux.

CRITIQUE Mutation d'une liste const — crash à l'exécution

lib/state/customer_orders_provider.dart (ligne 65)

`_orders = const []` initialise une liste non-mutable. La ligne `_orders[index] = order` tente de modifier cette liste et lève `Unsupported operation: indexed set` au runtime.

Correction : Initialiser avec `_orders = <Order>[]` (mutable) ou créer une copie : `final copy = [..._orders]; copy[index] = order; _orders = copy;`

● ARCHITECTURE & PERFORMANCE**ÉLEVÉ Formule haversine incorrecte dans la carte de livraison en direct**

lib/ui/widgets/client/live_delivery_map_sheet.dart (lignes 280-287)

Le calcul de distance utilise une formule simplifiée incorrecte au lieu de la vraie formule haversine. Les distances affichées au client sont erronées, notamment sur de longues distances.

Correction : Utiliser : $a = \sin^2(\Delta lat/2) + \cos(lat1) \cdot \cos(lat2) \cdot \sin^2(\Delta lng/2)$ puis $c = 2 \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a})$.

ÉLEVÉ Mutation d'état sans notifyListeners

lib/state/customer_orders_provider.dart (lignes 35-37)

Le cache des détails de commande est mis à jour (`_detailsCache[order.piece] = order`) sans appel à `notifyListeners()`. L'interface ne se met pas à jour après un changement de statut.

Correction : Ajouter `notifyListeners()`; après toute mutation d'état qui doit se répercuter dans l'interface.

ÉLEVÉ Logique métier dans les widgets (filtrage commandes)

lib/ui/screens/livreur/new_orders_screen.dart (lignes 49-65)

La logique de filtrage et de recherche (`_filter()`, `_matches()`) est implémentée dans le `StatefulWidget` au lieu du provider. Impossible à tester unitairement et couplée à l'UI.

Correction : Déplacer le filtrage vers `DeliveriesProvider.getFilteredNewOrders(String query)`. Le widget ne fait que consommer le résultat.

ÉLEVÉ Absence de gestion hors-ligne et de file de requêtes

lib/core/api_client.dart

Les erreurs réseau sont propagées mais aucune action n'est mise en file d'attente pour retry quand la connexion revient. Les livreurs en zone couverte partiellement perdent leurs mises à jour de statut.

Correction : Implémenter une file locale (Hive ou SQLite) pour les actions critiques (prise en charge, livraison). Retry automatique avec `connectivity_plus`.

MOYEN Rebond de polling inefficace — rebuilds inutiles

`lib/ui/screens/client_orders_screen.dart` (lignes 53-56)

La méthode `refresh()` est appelée toutes les 20 secondes, déclenchant un `notifyListeners()` même si les données n'ont pas changé. Tout l'arbre de widgets se reconstruit inutilement.

Correction : Comparer les nouvelles données aux actuelles avant `notifyListeners()`. Utiliser `Equatable` ou implémenter `==` sur les modèles.

MOYEN Exceptions SignalR avalées silencieusement

`lib/core/realtime_service.dart` (lignes 209-218)

Les échecs de connexion SignalR dans `ensureConnected()` ne sont pas remontés. L'application continue en mode "temps réel" sans connexion active, affichant des données périmées.

Correction : Logger les erreurs et exposer un état de connexion via le provider. Afficher un badge "hors-ligne" dans l'interface quand SignalR est déconnecté.

MOYEN Feedback haptique absent sur les actions critiques

`lib/ui/screens/livreur/new_orders_screen.dart`

Les actions critiques (prise en charge, livraison confirmée) n'ont pas de feedback haptique. Sur mobile, le retour vibratoire est essentiel pour confirmer les actions physiques.

Correction : Ajouter `HapticFeedback.mediumImpact()`; au début des handlers de prise en charge et de confirmation de livraison.

FAIBLE Logs de debug non désactivés en production

`lib/` (services multiples)

Des `print()` et `debugPrint()` sans garde `kDebugMode` émettent des logs en production, potentiellement exposant des informations sensibles dans les logcats des appareils.

Correction : Remplacer tous les `print()` par `if (kDebugMode) print()` ou utiliser un logger avec niveaux (logger package).



Plan de Remédiation

§ 5 — Actions prioritaires classées par impact et effort estimé

⚡ Sprint 1 — Actions immédiates (bloquantes en production)

1. **[Backend]** Corriger l'injection SQL dans `SetSageX3CustomerRefAsync` — risque légal et sécurité critique
2. **[Backend]** Envelopper la création de commande dans une transaction atomique — cohérence des données
3. **[Backend]** Valider les signatures du webhook paiement — risque de fraude immédiat
4. **[Flutter]** Corriger la mutation de liste const (`customer_orders_provider.dart`) — crash garanti en prod
5. **[Flutter]** Ajouter les vérifications mounted dans `login_screen` et `new_orders_screen` — crash sur navigation
6. **[React]** Désactiver les boutons pendant `isPending` pour éviter les doubles soumissions
7. **[React]** Ajouter des Error Boundaries globaux — empêcher le crash total de l'interface

🔧 Sprint 2 — Corrections élevées (dans les 30 jours)

1. **[Backend]** Ajouter `AsNoTracking()` sur toutes les requêtes en lecture seule — performance DB
2. **[Backend]** Résoudre les problèmes N+1 avec `.Include()` appropriés
3. **[Backend]** Implémenter la pagination sur les endpoints de listes
4. **[Backend]** Renforcer la politique de mot de passe + rate limiting auth
5. **[Flutter]** Implémenter certificate pinning en production
6. **[Flutter]** Ajouter vérification d'expiration JWT + refresh token
7. **[Flutter]** Remplacer `List.generate` par `ListView.builder` pour les grandes listes
8. **[React]** Migrer le token paiement hors de `sessionStorage` (cookies httpOnly)
9. **[React]** Corriger les dépendances `useEffect` (activer ESLint exhaustive-deps)

📋 Sprint 3 — Améliorations (dans les 60-90 jours)

1. **[Backend]** Ajouter un système de logging structuré avec `CorrelationId` (Serilog)
2. **[Backend]** Valider les DTO entrants avec `FluentValidation`

- 3. **[Flutter]** Implémenter la file hors-ligne pour les actions livreur
- 4. **[Flutter]** Centraliser la logique de filtrage dans les providers
- 5. **[Flutter]** Corriger la formule haversine dans la carte de livraison
- 6. **[React]** Ajouter la virtualisation des longues listes (react-virtual)
- 7. **[React]** Implémenter le code splitting avec React.lazy
- 8. **[React]** Ajouter les attributs ARIA d'accessibilité
- 9. **[Tout]** Mettre en place une suite de tests (Jest + RTL pour React, flutter_test pour Flutter)

PRIORITÉ	NOMBRE DE PROBLÈMES	IMPACT EN PRODUCTION	EFFORT ESTIMÉ
CRITIQUE Bloquant	19	Crashes, fraude, injection SQL	~2 semaines
ÉLEVÉ Important	40	Performance, sécurité, données incohérentes	~4 semaines
MOYEN Amélioration	48	UX, maintenabilité, accessibilité	~6 semaines
FAIBLE Optimisation	19	Qualité de code, documentation	~3 semaines

Rapport d'Audit Technique — Projet PFE 2026

Analyse conduite sur les trois couches applicatives : **Backend ASP.NET Core 8, Frontend React 18/TypeScript** et **Application Mobile Flutter 3**.

Ce document est confidentiel et destiné à l'équipe de développement du projet.

Généré le 09 juin 2026 · 126 problèmes identifiés sur l'ensemble du périmètre